

Aula 8 - Testes de Comportamento (BDD) com Cucumber

Duração: 2 horas

Objetivos:

- Compreender os princípios do Behavior-Driven Development (BDD)
- Escrever especificações executáveis em Gherkin
- Implementar testes BDD com Cucumber em Java

Parte Teórica (40 min)

1. O que é BDD?

Behavior-Driven Development (BDD) é uma metodologia que:

- Foca no **comportamento** do sistema, não na implementação
- Usa linguagem natural (Gherkin) para descrever testes
- Promove colaboração entre devs, QAs e POs

Diferença entre TDD e BDD:

TDD	BDD
"Como testar?"	"O que testar?"
Foco em unidades	Foco em features
Linguagem técnica	Linguagem natural

2. Componentes do Cucumber

1. **Arquivos** `.feature` : Especificações em Gherkin
2. **Step Definitions**: Código Java que implementa os passos
3. **Test Runner**: Classe que executa os testes

3. Sintaxe Gherkin

gherkin

Funcionalidade: Calculadora de soma

Como um usuário

Eu quero somar dois números

Para obter o resultado correto

Cenário: Soma de números positivos

Dado que eu tenho os números 2 e 3

Quando eu somo estes números

Então o resultado deve ser 5

Palavras-chave:

- Dado (Setup inicial)
- Quando (Ação)
- Então (Verificação)
- E , Mas (Para múltiplos passos)

Parte Prática (1h20min)

Atividade 1: Configurando Cucumber

1. Adicione as dependências no pom.xml :

xml

```
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>7.14.0</version>
</dependency>
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-junit</artifactId>
  <version>7.14.0</version>
</dependency>
```

2. Crie a estrutura de pastas:

text

```
src/test/
├─ java/
│   └─ steps/
│       └─ CalculadoraSteps.java
```

```
|   └─ runner/
|       └─ TestRunner.java
└─ resources/
    └─ features/
        └─ calculadora.feature
```

Atividade 2: Escrevendo a Primeira Feature

calculadora.feature:

gherkin

Funcionalidade: Calculadora

Cenário: Soma de dois números

Dado que eu tenho os números 5 e 7

Quando eu peço para somar

Então o resultado deve ser 12

Cenário: Divisão por zero

Dado que eu tenho os números 10 e 0

Quando eu peço para dividir

Então o sistema deve informar "Divisão por zero não permitida"

Atividade 3: Implementando Step Definitions

CalculadoraSteps.java:

java

```
public class CalculadoraSteps {
    private int num1, num2;
    private int resultado;
    private Exception exception;

    @Dado("que eu tenho os números {int} e {int}")
    public void definir_numeros(int a, int b) {
        this.num1 = a;
        this.num2 = b;
    }

    @Quando("eu peço para somar")
    public void somar() {
        this.resultado = Calculadora.somar(num1, num2);
    }
}
```

```

    @Então("o resultado deve ser {int}")
    public void verificar_resultado(int esperado) {
        assertEquals(esperado, resultado);
    }

    @Quando("eu peço para dividir")
    public void dividir() {
        try {
            this.resultado = Calculadora.dividir(num1, num2);
        } catch (Exception e) {
            this.exception = e;
        }
    }

    @Então("o sistema deve informar {string}")
    public void verificar_mensagem(String mensagem) {
        assertNotNull(exception);
        assertEquals(mensagem, exception.getMessage());
    }
}

```

Atividade 4: Configurando o Test Runner

TestRunner.java:

```

java

import org.junit.runner.RunWith;
import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "steps",
    plugin = {"pretty", "html:target/cucumber-reports.html"}
)
public class TestRunner {}

```

Exercício Prático (30 min)

Desafio: Implemente uma nova feature para teste de login:

gherkin

Funcionalidade: Autenticação de usuário

Cenário: Login válido

Dado que o usuário "admin" com senha "1234" está cadastrado

Quando eu faço login com "admin" e "1234"

Então o acesso é concedido

Cenário: Senha inválida

Dado que o usuário "admin" com senha "1234" está cadastrado

Quando eu faço login com "admin" e "senhaerrada"

Então o acesso é negado

Passos:

1. Crie `autenticacao.feature`
2. Implemente os step definitions
3. Execute via `TestRunner`

Encerramento

- Discussão: Quando usar BDD vs testes unitários tradicionais?
- Próxima aula: Versionamento e Git Avançado (Aula 9)

Tarefa:

- Escrever uma feature para o seu projeto final usando Gherkin
- Implementar pelo menos 2 cenários

Recursos:

- [Cucumber Documentation](#)
- [Gherkin Reference](#)

Prontos para transformar requisitos em testes executáveis? Dúvidas? 😊🚀